C++ Standard Template Library (STL)

CodeIIEST Summer Bootcamp 2026

Week 01 · Prerequisites

Contents

1. Introduction

The C++ Standard Template Library (STL) provides generic classes and functions. It consists of four core components:

- **Containers**: Objects that store data collections.
- **Iterators**: Objects that act like pointers to elements inside containers.
- **Algorithms**: Functions that perform operations (sort, search, math) on containers.
- **Functors (Function Objects)**: Classes that act like functions.

2. How STL Groups Data

To effectively use the STL, it helps to understand how it organizes data into groups. You can think of data in three levels:

- **Basic Variables**: The simplest building blocks, like a single `int`, `char`, or `float`.
- **Small Groups (Utilities)**: Structures that group a few variables together into a single unit. For example, a `pair` groups exactly two variables (even of different types), and a `tuple` groups several variables.
- **Large Groups (Containers)**: Large structures designed to hold many items. The beauty of the STL is that containers don't just hold basic variables—they can hold small groups, or even other containers.

By combining these, you can build powerful setups with a single line of code.

Example of Grouping in Action:

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    // 1. Basic Variables
    int age = 20;
    string name = "Alice";

    // 2. Small Group (Grouping basic variables)
    pair<int, string> student = {age, name};

    // 3. Large Group (Grouping small groups together)
    vector<pair<int, string>> classroom;
    classroom.push_back(student);

    return 0;
}
```

3. Utilities

Small, helpful structures used heavily throughout the STL to group variables.

3.1 pair

Concept

Stores two pieces of data of different types as a single unit.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    pair<int, string> p1 = {1, "Apple"};
    pair<int, string> p2 = make_pair(2, "Banana");

    cout << p1.first << " - " << p1.second << "\n";
    return 0;
}
```

3.2 tuple

Concept

Like a `pair`, but can hold any number of elements.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    tuple<int, double, char> t = {1, 3.14, 'A'};
    cout << get<0>(t) << ", " << get<1>(t) << "\n";

    // Modify value
    get<2>(t) = 'B';
    return 0;
}
```

***Note**: Utilities like `pair` and `tuple` are incredibly powerful because they can be stored directly inside containers. You can easily create complex structures like a `vector<pair<int, int>>`, a `map<string, tuple<int, double, char>>`, or a `set<pair<int, string>>`.*

4. Sequence Containers

Containers that store data in a linear manner.

4.1 vector

Concept

Dynamic array. Fast random access. Slow insertion/deletion at the front or middle. Fast at the back.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3};

    v.push_back(4);    // Adds 4 to end: {1, 2, 3, 4}
    v.pop_back();      // Removes last element: {1, 2, 3}

    int size = v.size();       // Number of elements
    int first = v.front();     // First element (1)
    int last = v.back();       // Last element (3)

    v.clear(); // Empties the vector
    return 0;
}
```

4.2 list

Concept

Doubly linked list. Fast insertion/deletion anywhere. Slow access (no random access via [ ]).

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    list<int> l = {2, 3};

    l.push_front(1); // O(1) insertion at front: {1, 2, 3}
    l.push_back(4);  // O(1) insertion at back: {1, 2, 3, 4}

    l.pop_front();   // Removes 1
    l.reverse();     // Reverses the list
    return 0;
}
```

4.3 deque

Concept

Double-ended queue. Dynamic arrays structured in chunks. Fast insertion/deletion at **both** ends.

***Note**: We will discuss this in deeper detail later in the bootcamp.*

5. Container Adaptors

Interfaces built on top of sequence containers to provide specific behavior. No iterators allowed.

5.1 stack

Concept

LIFO (Last In, First Out) data structure.

***Note**: We will discuss this in deeper detail later in the bootcamp.*

5.2 queue

Concept

FIFO (First In, First Out) data structure.

***Note**: We will discuss this in deeper detail later in the bootcamp.*

5.3 priority_queue

Concept

Max-Heap by default. Largest element is always at the top.

***Note**: We will discuss this in deeper detail later in the bootcamp.*

6. Associative Containers

Containers that store data in a sorted manner (typically implemented via Red-Black Trees). Search, insertion, and deletion are $O(\log N)$.

6.1 set

Concept

Stores **unique** elements in **sorted** order.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    set<int> s = {3, 1, 4, 1}; // Stores as {1, 3, 4}

    s.insert(5);      // Adds 5: {1, 3, 4, 5}
    s.erase(3);       // Removes 3: {1, 4, 5}

    // 1. Iterating through a set
    for (auto it = s.begin(); it != s.end(); ++it) {
        cout << *it << " ";
    }

    // 2. Finding elements
    if (s.find(4) != s.end()) {
        // 4 exists
    }

    // count() also works: returns 1 if exists, 0 otherwise
    if (s.count(5)) {
        // 5 exists
    }

    return 0;
}
```

***Note**: A* `multiset` *allows duplicate elements while maintaining order.*

6.2 map

Concept

Stores key-value pairs. Keys are **unique** and **sorted**.

```cpp
#include <bits/stdc++.h>
using namespace std;
```

```cpp

int main() {
    map<string, int> m;

    m["apple"] = 5;       // Insertion/Update
    m.insert({"banana", 3});

    // 1. Iterating through a map using iterators
    for (auto it = m.begin(); it != m.end(); ++it) {
        // it->first is the key, it->second is the value
        cout << it->first << " : " << it->second << "\n";
    }

    // Modern, easier approach for iteration
    for (auto p : m) {
        cout << p.first << " : " << p.second << "\n";
    }

    // 2. Finding keys
    if (m.find("apple") != m.end()) {
        cout << "Found apple!";
    }

    m.erase("banana"); // Removes by key
    return 0;
}
```

***Note**: A* `multimap` *allows duplicate keys.*

7. Unordered Containers

Implemented using Hash Tables. Keys are **not sorted**. Search, insertion, and deletion are $O(1)$ on average.

7.1 unordered_set & unordered_map

Concept

Same as `set` and `map`, but sacrifices ordering for speed.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    unordered_set<int> uset = {5, 1, 9}; // Order in memory is random

    // Element is placed into a "bucket" computed from its hash value.
    // It does NOT go to the 'end' or 'front' like in a vector or list.
    uset.insert(4);
```

```cpp

    unordered_map<int, string> umap;
    umap[1] = "One";
    umap[2] = "Two";

    // Syntax for checking existence is identical to ordered versions
    if (umap.find(1) != umap.end()) {
        // Exists
    }
    return 0;
}
```

8. Iterators

Iterators act like pointers pointing to elements inside STL containers.

Key Functions:

- `begin()`: Points to the first element.
- `end()`: Points to the theoretical element **after** the last element.
- `rbegin()`: Points to the last element (used for reverse iteration).
- `rend()`: Points to the theoretical element before the first element.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> v = {10, 20, 30};

    // Forward iteration
    for (auto it = v.begin(); it != v.end(); ++it) {
        cout << *it << " "; // Dereference iterator to get value
    }

    // Reverse iteration
    for (auto it = v.rbegin(); it != v.rend(); ++it) {
        cout << *it << " ";
    }

    // Modern Range-Based For Loop (Uses begin() and end() under the hood)
    for (int val : v) {
        cout << val << " ";
    }
    return 0;
}
```

9. Algorithms

Powerful built-in functions that operate on ranges (using iterators).

9.1 Sorting & Searching

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> v = {4, 1, 3, 5, 2};

    // 1. Sort (Ascending by default) O(N log N)
    sort(v.begin(), v.end()); // v becomes {1, 2, 3, 4, 5}

    // 2. Sort Descending using greater functor
    sort(v.begin(), v.end(), greater<int>());

    // 3. Binary Search (Requires sorted container) O(log N)
    sort(v.begin(), v.end()); // sort again
    bool found = binary_search(v.begin(), v.end(), 3); // Returns true

    // 4. Lower Bound (First element >= target)
    auto lb = lower_bound(v.begin(), v.end(), 3);
    // *lb is 3

    // 5. Upper Bound (First element > target)
    auto ub = upper_bound(v.begin(), v.end(), 3);
    // *ub is 4

    return 0;
}
```

9.2 General Functions

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> v = {1, 2, 2, 3, 4, 5};

    // 1. Count occurrences
    int twos = count(v.begin(), v.end(), 2); // Returns 2

    // 2. Reverse a range
    reverse(v.begin(), v.end()); // {5, 4, 3, 2, 2, 1}

    // 3. Max and Min Element
    auto max_it = max_element(v.begin(), v.end()); // points to 5
    auto min_it = min_element(v.begin(), v.end()); // points to 1
```

```
    // 4. Accumulate (Sum of elements)
    // 0 is the initial sum value
    int sum = accumulate(v.begin(), v.end(), 0); // Returns 17

    // 5. Next Permutation (Finds the next lexicographically greater permutation)
    vector<int> p = {1, 2, 3};
    next_permutation(p.begin(), p.end()); // p becomes {1, 3, 2}

    // 6. fill: Puts the same value in the entire range
    vector<int> v2(5);
    fill(v2.begin(), v2.end(), -1); // {-1, -1, -1, -1, -1}

    // 7. iota: Fills the range with sequentially increasing values
    iota(v2.begin(), v2.end(), 10); // {10, 11, 12, 13, 14}

    return 0;
}
```

9.3 Custom Sorting

Concept

You can define your own rules for sorting by passing a custom comparison function (comparator).

```
#include <bits/stdc++.h>
using namespace std;

// Custom comparator to sort in descending order
bool myComparator(int a, int b) {
    return a > b;
}

// Custom comparator to sort pairs by their second element
bool sortPairs(pair<int, int> a, pair<int, int> b) {
    return a.second < b.second;
}

int main() {
    vector<int> v = {1, 4, 2, 5};
    sort(v.begin(), v.end(), myComparator); // {5, 4, 2, 1}

    vector<pair<int, int>> vecPairs = {{1, 3}, {2, 1}, {3, 2}};
    sort(vecPairs.begin(), vecPairs.end(), sortPairs);
    // Becomes: {{2, 1}, {3, 2}, {1, 3}}

    return 0;
}
```

10. Time Complexity Reference

Understanding the underlying time complexity of various operations is crucial for choosing the right container for your algorithms. Below is a summary of the Big-O complexities for common operations.

Container	Access (by index)	Insert/Delete (Front)	Insert/Delete (Back)	Search (by value/key)
vector	$O(1)$	$O(N)$	$O(1)$ (amortized)	$O(N)$
list	$O(N)$	$O(1)$	$O(1)$	$O(N)$
deque	$O(1)$	$O(1)$	$O(1)$	$O(N)$
set / map	N/A	$O(\log N)$ (Insert Anywhere)		$O(\log N)$
unordered_set/map	N/A	$O(1)$ avg (Insert Anywhere)		$O(1)$ avg
stack	$O(1)$ (Top only)	N/A	$O(1)$	N/A
queue	$O(1)$ (Front/Back)	$O(1)$	$O(1)$	N/A
priority_queue	$O(1)$ (Top only)	N/A	$O(\log N)$	N/A

***Note**: "Access" refers to randomly accessing an element via the* `[]` *operator. "Search" refers to locating an element by its value (or by key in maps/sets). While* `unordered` *containers are $O(1)$ on average, their worst-case complexity can degrade to $O(N)$ during hash collisions.*

11. Recommended Resources

For further practice and deeper understanding, here are the top curated resources highly recommended by the competitive programming community:

- **USACO Guide**: The absolute gold standard for structured CP learning.
  - Bronze: Introduction to Data Structures (STL)
- **GeeksforGeeks (GFG)**: Best as a dictionary for quick syntax lookups.
  - C++ Standard Template Library (STL) Crash Course
  - std::sort & Custom Comparators in C++
- **YouTube Channels**: Best for visual learners and deeper explanations.
  - Luv's CP Playlist (Excellent deep dives into STL, Maps, and Sets)
  - Take U Forward / Striver (Rapid "C++ STL in 1 Video" crash course)